損保アプリ AWSデプロイ手順書

この手順書について

損保業務支援アプリ（InsuranceApp-SpringBoot）をAWSにデプロイし、インターネットからアクセスできるようにする手順書です。 EC2パターンとLightsailパターンの2つを記載しています。どちらか好みの方を選んで実施してください。

前提条件

- AWSアカウントを持っていること（無料枠でOK）
- ローカルPCでDocker動作確認済み（Day97完了済み）
- Git/GitHubにプロジェクトをpush済み
- ローカルPCにSSHクライアントがあること（Windows: PowerShell / Mac: ターミナル）

EC2とLightsailの比較

項目	EC2	Lightsail
料金体系	従量課金（使った分だけ）	月額固定（$3.50〜）
設定の自由度	高い（VPC/サブネットも自分で設定可能）	低い（シンプルだが制約あり）
無料枠	t2.micro 12ヶ月無料	最初の3ヶ月 $0（$3.50プラン）
推奨する人	AWSの仕組みを学びたい人	すぐに公開したい人
セットアップ難度	やや高い	低い

おすすめ: ポートフォリオ用途なら**Lightsail**が手軽。AWSの学習も兼ねるなら**EC2**。

パターンA: EC2でデプロイ

A-1. EC2インスタンスの作成

1. AWSマネジメントコンソール（https://console.aws.amazon.com）にログイン
2. 画面上部の検索バーに「EC2」と入力し、**EC2**をクリック
3. 左メニューの**インスタンス**をクリック
4. **インスタンスを起動**ボタンをクリック

設定項目

設定項目	入力値
名前	insurance-app-server
AMI（OS）	Amazon Linux 2023 AMI（デフォルトでOK）
インスタンスタイプ	t2.micro（無料枠対象）

設定項目	入力値
キーペア	「新しいキーペアを作成」→ 名前: insurance-key → RSA → .pem → 「キーペアを作成」

重要: ダウンロードされた `insurance-key.pem` は絶対になくさないでください。これがサーバーへの「鍵」です。

ネットワーク設定（セキュリティグループ）

「編集」をクリックして以下を設定:

ルール	タイプ	ポート範囲	ソース	用途
ルール1	SSH	22	マイIP	サーバーに接続するため
ルール2	カスタムTCP	8080	0.0.0.0/0	アプリにアクセスするため
ルール3	HTTP	80	0.0.0.0/0	（将来リバースプロキシ用）

セキュリティグループ名: `insurance-app-sg`

ストレージ

- 20 GiB（デフォルト8GiBだとDockerビルドで足りない可能性があるため増やす）

5. **インスタンスを起動**をクリック
6. 起動完了後、インスタンス一覧で**パブリック IPv4 アドレス**をメモする

A-2. Elastic IP の割り当て（推奨）

EC2はデフォルトで再起動するとIPアドレスが変わります。固定したい場合はElastic IPを設定します。

1. EC2ダッシュボード → 左メニュー「ネットワーク＆セキュリティ」→**Elastic IP**
2. **Elastic IP アドレスの割り当て**をクリック
3. そのまま**割り当て**をクリック
4. 作成されたElastic IPを選択 → **アクション** → **Elastic IPアドレスの関連付け**
5. インスタンスで `insurance-app-server` を選択 → **関連付ける**

注意: Elastic IPは、EC2に関連付けていれば無料。関連付けずに放置すると課金されます。

A-3. SSH接続

ローカルPCのターミナル（PowerShell / ターミナル）で実行:

```
# Windowsの場合: insurance-key.pemがダウンロードフォルダにある前提
cd C:\Users\<あなたのユーザー名>\Downloads

# .pemファイルのアクセス権を設定（初回のみ）
# Windowsの場合:
icacls insurance-key.pem /inheritance:r /grant:r "%USERNAME%:R"
```

```
# Mac/Linuxの場合:
# chmod 400 insurance-key.pem

# SSH接続
ssh -i insurance-key.pem ec2-user@<EC2のパブリックIP>
```

「Are you sure you want to continue connecting?」と聞かれたら `yes` と入力。

接続成功すると、プロンプトが `[ec2-user@ip-xxx-xxx-xxx-xxx ~]$` に変わります。

A-4. サーバーセットアップ

方法1: セットアップスクリプトを使う場合

ローカルPCから別のターミナルを開いて:

```
# スクリプトをサーバーに送る
scp -i insurance-key.pem deploy/aws-ec2-setup.sh ec2-user@<IP>:~/

# サーバー上で実行
ssh -i insurance-key.pem ec2-user@<IP>
chmod +x ~/aws-ec2-setup.sh
./aws-ec2-setup.sh
```

方法2: 手動で実行する場合

```
# パッケージ更新
sudo dnf update -y

# Dockerインストール
sudo dnf install -y docker
sudo systemctl start docker
sudo systemctl enable docker
sudo usermod -aG docker ec2-user

# Docker Composeインストール（プラグイン版）
sudo mkdir -p /usr/local/lib/docker/cli-plugins/
sudo curl -SL "https://github.com/docker/compose/releases/latest/download/docker-compose-linux-$(uname -m)" \
  -o /usr/local/lib/docker/cli-plugins/docker-compose
sudo chmod +x /usr/local/lib/docker/cli-plugins/docker-compose

# Gitインストール
sudo dnf install -y git

# 確認
docker --version
docker compose version
```

セットアップ後、**一度ログアウトして再接続**してください（dockerグループ反映のため）：

```
exit
ssh -i insurance-key.pem ec2-user@<IP>
```

A-5. プロジェクトファイルの転送と起動

方法1: SCPで転送

ローカルPCから:

```
# プロジェクト全体を送る（.gitやtargetは除外推奨）
scp -i insurance-key.pem -r ./insurance-app/ ec2-user@<IP>:~/
```

方法2: GitHubからclone（推奨）

サーバー上で:

```
cd ~
git clone https://github.com/<あなたのユーザー名>/InsuranceApp-SpringBoot.git
insurance-app
```

.envファイルの作成

```
# テンプレートをコピー
cp ~/insurance-app/deploy/.env.template ~/insurance-app/docker/.env

# 値を編集（nanoエディタが開く）
nano ~/insurance-app/docker/.env
```

nanoエディタの操作:

- 矢印キーでカーソル移動
- 「ここに〜」の部分を実際の値に書き換える
- **Ctrl + O** → Enter で保存
- **Ctrl + X** で終了

アプリの起動

```
cd ~/insurance-app/deploy
docker compose -f docker-compose.prod.yml up -d --build
```

初回ビルドは10〜15分かかります。進捗を見るには:

```
docker compose -f docker-compose.prod.yml logs -f app
```

以下のログが表示されれば起動成功:

```
Started InsuranceApplication in X.XXX seconds
```

Ctrl + C でログ表示を終了。

A-6. 動作確認

ブラウザで以下にアクセス:

```
http://<EC2のパブリックIP>:8080
```

損保アプリのホーム画面が表示されれば成功です。

パターンB: Lightsailでデプロイ

B-1. Lightsailインスタンスの作成

1. https://lightsail.aws.amazon.com にアクセス
2. **インスタンスの作成**をクリック

設定項目

設定項目	入力値
リージョン	東京（ap-northeast-1）
プラットフォーム	Linux/Unix
設計図の選択	「OSのみ」→ **Ubuntu 22.04 LTS**
インスタンスプラン	$5 USD/月（1GB RAM, 1 vCPU）
インスタンス名	insurance-app-server

注意: $3.50プランだとメモリ512MBで、Dockerビルドに不足する可能性があります。$5プランを推奨。

3. **インスタンスの作成**をクリック

B-2. 静的IPの割り当て

1. 作成したインスタンスの管理画面を開く
2. **ネットワーキング**タブ → **静的IPの作成**

3. 名前: `insurance-app-ip` → **作成**

B-3. ファイアウォール設定（ポート開放）

1. インスタンスの管理画面 → **ネットワーキング**タブ
2. **IPv4 ファイアウォール**セクションで**ルールの追加**:

アプリケーション	プロトコル	ポート
SSH	TCP	22（デフォルトで開いている）
カスタム	TCP	8080
HTTP	TCP	80

B-4. SSH接続

方法1: ブラウザSSH（簡単）

1. Lightsailのインスタンス一覧で、インスタンス名の右にあるターミナルアイコンをクリック
2. ブラウザ上でターミナルが開く

方法2: ローカルPCから接続

1. Lightsail管理画面 → **アカウント** → **SSH キー** → デフォルトキーをダウンロード
2. ローカルPCのターミナルで:

```
chmod 400 LightsailDefaultKey-ap-northeast-1.pem
ssh -i LightsailDefaultKey-ap-northeast-1.pem ubuntu@<LightsailのパブリックIP>
```

B-5. サーバーセットアップ

方法1: セットアップスクリプトを使う場合

ローカルPCから:

```
scp -i LightsailDefaultKey-ap-northeast-1.pem deploy/aws-lightsail-setup.sh
ubuntu@<IP>:~/
ssh -i LightsailDefaultKey-ap-northeast-1.pem ubuntu@<IP>
chmod +x ~/aws-lightsail-setup.sh
./aws-lightsail-setup.sh
```

方法2: 手動で実行する場合

```
# パッケージ更新
sudo apt update -y && sudo apt upgrade -y

# Docker公式リポジトリ追加 & インストール
```

```
sudo apt install -y ca-certificates curl gnupg
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
/etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg
echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt update -y
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin
sudo systemctl start docker
sudo systemctl enable docker
sudo usermod -aG docker ubuntu

# 確認
docker --version
docker compose version
```

一度ログアウトして再接続:

```
exit
# 再接続
```

B-6. プロジェクトファイルの転送と起動

A-5と同じ手順です（ユーザー名が `ubuntu` になる点だけ注意）。

```
# GitHubからclone（推奨）
cd ~
git clone https://github.com/<あなたのユーザー名>/InsuranceApp-SpringBoot.git
insurance-app

# .env作成
cp ~/insurance-app/deploy/.env.template ~/insurance-app/docker/.env
nano ~/insurance-app/docker/.env

# 起動
cd ~/insurance-app/deploy
docker compose -f docker-compose.prod.yml up -d --build
```

B-7. 動作確認

```
http://<LightsailのパブリックIP>:8080
```

共通: 運用コマンド集

アプリの状態確認

```
# コンテナの状態確認
docker compose -f docker-compose.prod.yml ps

# 期待される出力:
# NAME                STATUS          PORTS
# insurance-app       Up (healthy)    0.0.0.0:8080->8080/tcp
# insurance-mysql     Up (healthy)    3306/tcp
```

ログの確認

```
# アプリのログ（直近100行）
docker compose -f docker-compose.prod.yml logs --tail=100 app

# リアルタイムでログを追う
docker compose -f docker-compose.prod.yml logs -f app

# MySQLのログ
docker compose -f docker-compose.prod.yml logs --tail=50 mysql
```

アプリの再起動

```
# 停止 → 起動
docker compose -f docker-compose.prod.yml restart

# コードを更新した場合（再ビルドが必要）
docker compose -f docker-compose.prod.yml up -d --build
```

アプリの停止

```
# 停止（データは残る）
docker compose -f docker-compose.prod.yml down

# 停止 + データも削除（完全クリーン）
docker compose -f docker-compose.prod.yml down -v
```

コードの更新（デプロイ更新）

```
cd ~/insurance-app
```

```
# 最新コードを取得
git pull origin main

# 再ビルド＆再起動
cd deploy
docker compose -f docker-compose.prod.yml up -d --build
```

トラブルシューティング

症状: SSH接続できない

原因	対処
セキュリティグループでポート22が開いていない	AWSコンソールでインバウンドルールにSSH(22)を追加
.pemファイルの権限が緩すぎる	`chmod 400 <key>.pem`（Mac/Linux）または icacls で制限（Windows）
IPアドレスが間違っている	AWSコンソールでパブリックIPを再確認

症状: ブラウザでアクセスできない

原因	対処
セキュリティグループでポート8080が開いていない	インバウンドルールにカスタムTCP 8080を追加
コンテナが起動していない	`docker compose -f docker-compose.prod.yml ps` で状態確認
ビルドに失敗している	`docker compose -f docker-compose.prod.yml logs app` でエラー確認

症状: アプリが起動するがDBに接続できない

原因	対処
.envのDB接続情報が間違っている	.envのSPRING_DATASOURCE_URL等を確認
MySQLコンテナが起動していない	`docker compose -f docker-compose.prod.yml logs mysql` で確認
MySQLのヘルスチェックが通らない	60秒待ってから再確認（起動に時間がかかる場合がある）

症状: Dockerビルドが途中で失敗する

原因	対処

原因	対処
メモリ不足（512MBプランなど）	インスタンスのプランを$5以上にアップグレード
ディスク容量不足	`df -h` で確認。古いDockerイメージを `docker system prune -a` で削除
mvnwに実行権限がない	`chmod +x mvnw` をプロジェクトルートで実行

費用の目安

サービス	プラン	月額費用
EC2 (t2.micro)	無料枠（12ヶ月）	$0
EC2 (t2.micro)	無料枠終了後	約$8〜10
Lightsail	$5プラン	$5
Elastic IP	EC2に関連付け済み	$0
Elastic IP	未関連付け	約$3.65/月

ポートフォリオ用途のコツ: 面接期間中のみ起動し、終わったら停止すれば費用を抑えられます。

チェックリスト

- ☐ AWSアカウント作成済み
- ☐ EC2 or Lightsailインスタンス作成済み
- ☐ セキュリティグループ/ファイアウォールでポート22, 8080を開放済み
- ☐ SSH接続成功
- ☐ Docker & Docker Compose インストール済み
- ☐ プロジェクトファイルをサーバーに配置済み
- ☐ .envファイルを実際の値で作成済み
- ☐ docker compose up -d --build で起動成功
- ☐ ブラウザで http://:8080 にアクセスしてホーム画面表示
- ☐ Google OAuth のリダイレクトURIにサーバーのURLを追加済み
- ☐ .envファイルが.gitignoreに含まれていることを確認